

BitPub Documento di Implementazione

1. Descrizione e motivazione dell'uso dei pattern

Per il progetto BitPub, oltre all'architettura generale basata su microservizi distribuiti, abbiamo implementato alcuni Design Pattern specifici ("GoF Patterns") per risolvere problemi ricorrenti legati alla comunicazione di rete asincrona e alla gestione degli eventi IoT.

A. Pattern Command

- Classe: `it.uniupo.pissir.bitpub.common.mqtt.MqttCommandWrapper`
- Tipologia: Comportamentale

Descrizione: Il pattern Command garantisce che ogni richiesta o azione generata dal sistema sia incapsulata all'interno di un oggetto strutturato. Nel nostro progetto, la classe `MqttCommandWrapper` è stata progettata in modo che tutte le direttive inviate verso i dispositivi fisici contengano un payload, un identificatore e una tipologia di azione ben standardizzati.

Motivazione dell'uso: L'interazione tra l'infrastruttura Cloud e i nodi Edge (tramite un broker MQTT) richiede un elevato livello di disaccoppiamento e robustezza. Trasmettere istruzioni non strutturate rischierebbe di generare errori di decodifica e renderebbe complessa la gestione delle code di messaggi. Il pattern Command ci assicura che l'intera applicazione utilizzi un formato univoco e condiviso per richiedere operazioni remote (come l'avvio o l'interruzione di una partita), centralizzando la serializzazione delle comunicazioni IoT.

B. Pattern Observer (Publish-Subscribe)

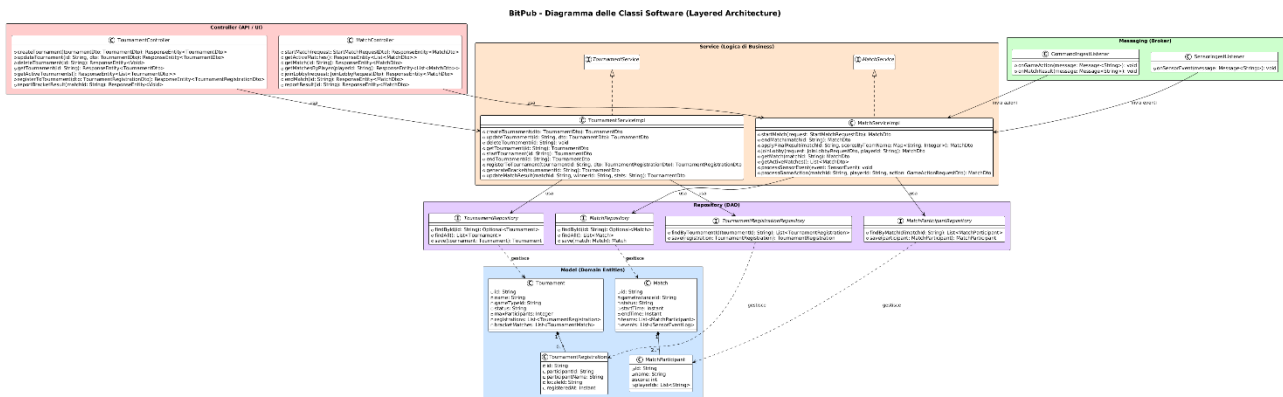
- Package: `it.uniupo.pissir.bitpub.matchservice.config`
- Tipologia: Comportamentale

Descrizione: Abbiamo definito classi listener specifiche, come `SensorIngestListener`, che dichiarano la loro sottoscrizione a determinati topic di rete. Queste classi restano in attesa passiva di eventi asincroni e reagiscono eseguendo logiche specifiche nel momento esatto in cui i dispositivi pubblicano nuovi aggiornamenti.

Motivazione dell'uso: Questo pattern è stato fondamentale per disaccoppiare la generazione dei dati (ovvero le rilevazioni effettuate dai sensori fisici ai margini della rete) dalle logiche Cloud di avanzamento e validazione della partita. I singoli Service non effettuano costose e inefficienti chiamate di polling per verificare lo stato dell'hardware; al contrario, il sistema reagisce dinamicamente intercettando gli input tramite i listener. Questo rende l'infrastruttura estremamente reattiva, manutenibile e pronta a gestire picchi di messaggistica qualora il numero di terminali fisici dovesse crescere.

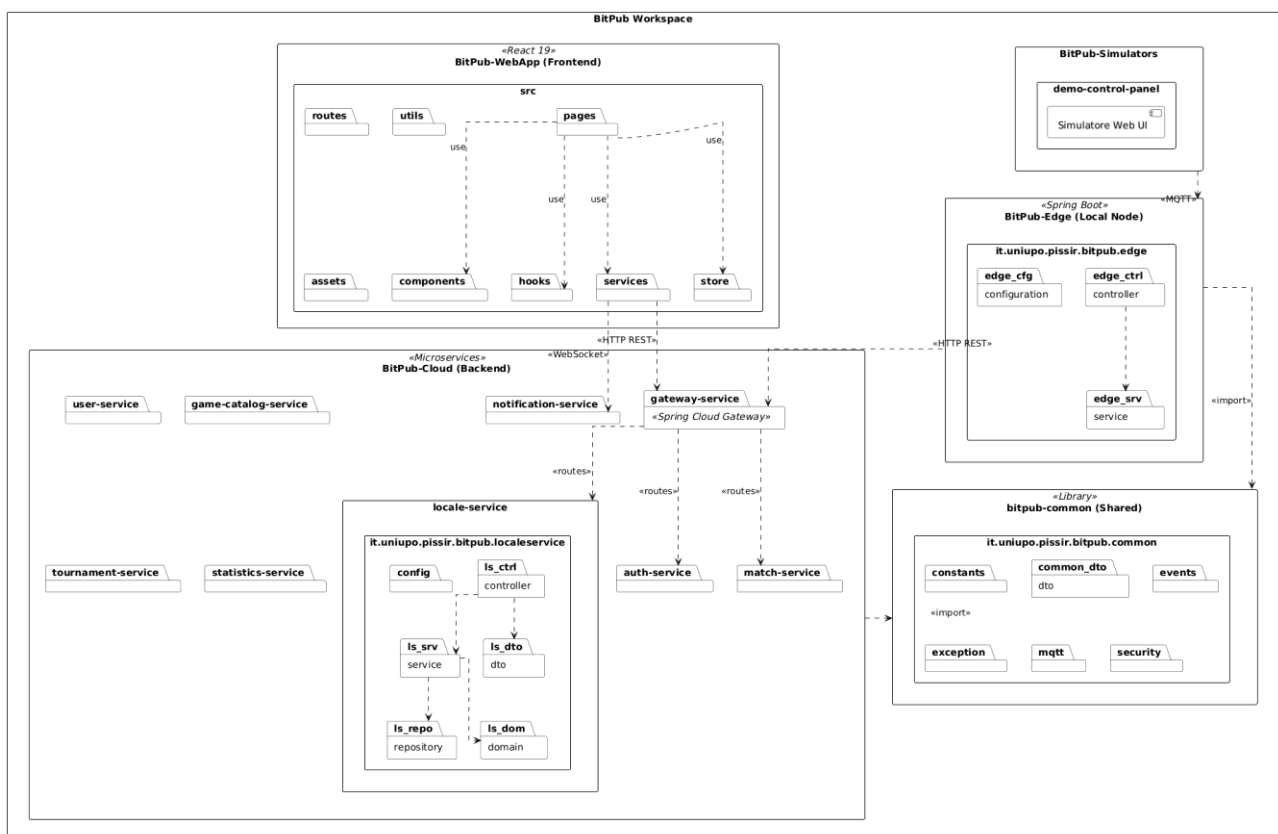
2. Diagramma Classi Software

Il seguente diagramma illustra la struttura statica e di dominio del sistema BitPub, mettendo in evidenza entità chiave come Match, Tournament, User e SensorEventLog, unitamente ai controller, i DTO e le logiche di servizio che implementano le regole di gioco.



3. Diagramma dei Package

Questo schema rappresenta l'organizzazione logica e modulare del progetto. Il diagramma delinea i vari microservizi isolati (es. auth-service, match-service, gateway-service), il componente Edge locale e la libreria condivisa bitpub-common utilizzata per mantenere l'uniformità dei contratti.



4. Diagramma di deployment

Il diagramma di deployment descrive la complessa topologia infrastrutturale della piattaforma. Vengono evidenziati i container Docker che ospitano i microservizi nel Cloud, i database isolati, l'istanza del broker MQTT e l'ambiente applicativo distribuito sui dispositivi fisici remoti (Edge).

